ASSIGNMENT-11.2

Roll no:2303A52074
Batch-37

Task Description -1 – (Stack Using AI Guidance)

Prompt

Create a Stack class in with basic operations like push, pop, peek, and checking if the stack is empty as user defined input.

Code

ai 11.py > ...

```python
#Create a Stack class in with basic operations like push, pop, peek, and checking if the stack is empty as user defined input.
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.stack.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack. Raises an error if the stack is empty."""
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        return self.stack.pop()

    def peek(self):
        """Return the item at the top of the stack without removing it. Raises an error if the stack is empty."""
        if self.is_empty():
            raise IndexError("Peek from an empty stack")
        return self.stack[-1]

    def is_empty(self):
        """Check if the stack is empty."""
        return len(self.stack) == 0
# Example usage:
stack = Stack()
stack.push(1)
stack.push(2)
print(stack.peek())  # Output: 2
print(stack.pop())   # Output: 2
print(stack.is_empty())  # Output: False
print(stack.pop())   # Output: 1
print(stack.is_empty())  # Output: True
```

Output

```
2
2
False
1
True
```

Explanation

In this task, we use AI to create a Stack class in Python.
The stack follows the LIFO principle.
It includes basic operations like push, pop, peek, and checking if it is empty.
Error handling is added to prevent removing elements from an empty stack.

Task Description -2 – (Queue Design)

Prompt

Create a simple Queue class in Python using a list that follows FIFO and includes enqueue, dequeue, front, size, and is empty methods with example usage.

Code

```python
#Create a simple Queue class in Python using a list that follows FIFO and includes enqueue, dequeue, front, size, and is_empty methods wit
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        """Add an item to the end of the queue."""
        self.queue.append(item)

    def dequeue(self):
        """Remove and return the item at the front of the queue. Raises an error if the queue is empty."""
        if self.is_empty():
            raise IndexError("Dequeue from an empty queue")
        return self.queue.pop(0)

    def front(self):
        """Return the item at the front of the queue without removing it. Raises an error if the queue is empty."""
        if self.is_empty():
            raise IndexError("Front from an empty queue")
        return self.queue[0]

    def size(self):
        """Return the number of items in the queue."""
        return len(self.queue)

    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.queue) == 0
            return self.queue[0]

        def size(self):
            """Return the number of items in the queue."""
            return len(self.queue)

        def is_empty(self):
            """Check if the queue is empty."""
            return len(self.queue) == 0
    # Example usage:
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    print(queue.front())  # Output: 1
    print(queue.dequeue())  # Output: 1
    print(queue.size())  # Output: 1
    print(queue.is_empty())  # Output: False
    queue.dequeue()  # Output: 2
    print(queue.is_empty())  # Output: True '''
```

Output

```
1
1
1
False
True
```

Explanation

We created a Queue class in Python using a list to store elements.

The queue follows FIFO, where the first inserted element is removed first.We implemented basic operations like enqueue, dequeue, front, size, and is empty to manage the queue.

Task Description -3 –(Singly Linked List Construction)

Prompt

Create a simple Singly Linked List in Python that includes a Node class, insertion method, and traversal function.

Code

```python
#Create a simple Singly Linked List in Python that includes a Node class, insertion method, and traversal functi
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        """Insert a new node with the given data at the end of the list."""
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        """Traverse the list and print the data of each node."""
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
```

```python
    def display(self):
        """Traverse the list and print the data of each node."""
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()  # for a new line after displaying the list
# Example usage:
linked_list = SinglyLinkedList()
linked_list.insert(1)
linked_list.insert(2)
linked_list.insert(3)
linked_list.display()  # Output: 1 2 3
```

Output

```
1 2 3
```

Explanation

We created a Node class to store data and a reference to the next node.
Then we built a Singly Linked List class to insert new nodes into the list.
Finally, we implemented a display function to print all elements from head to end.

Task Description -4 – (Binary Search Tree Operations)

Prompt

Create a Binary Search Tree in Python with a Node class, insertion method, and in-order traversal function.

Code

```python
#Create a Binary Search Tree in Python with a Node class, insertion method, and in-order traversal function.
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        """Insert a new node with the given data into the BST."""
        if self.root is None:
            self.root = TreeNode(data)
        else:
            self._insert_recursive(self.root, data)

    def _insert_recursive(self, node, data):
        if data < node.data:
            if node.left is None:
                node.left = TreeNode(data)
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = TreeNode(data)
            else:
    def _insert_recursive(self, node, data):
                self._insert_recursive(node.right, data)

    def in_order_traversal(self):
        """Perform in-order traversal of the BST and return a list of values."""
        return self._in_order_recursive(self.root)

    def _in_order_recursive(self, node):
        result = []
        if node:
            result.extend(self._in_order_recursive(node.left))
            result.append(node.data)
            result.extend(self._in_order_recursive(node.right))
        return result
# Example usage:
bst = BinarySearchTree()
bst.insert(5)
bst.insert(3)
bst.insert(7)
print(bst.in_order_traversal())  # Output: [3, 5, 7]
```

Output

```
[3, 5, 7]
```

Explanation

We created a Node class to store data and references to left and right child nodes.
Then we implemented insertion logic that places smaller values on the left and larger values on the right following BST rules.
Finally, we used in-order traversal to print the elements in sorted order.

Task Description -5 – (Hash Table Implementation)

Prompt

Create a Hash Table in Python that supports insert, search, and delete operations using collision handling.

Code

```python
#Create a Hash Table in Python that supports insert, search, and delete operations using collision handling
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate a hash for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)  # Update existing key
                return
        self.table[index].append((key, value))  # Insert new key-value pair

    def search(self, key):
        """Search for a value by its key. Returns the value or None if not found."""
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None  # Key not found

    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]  # Remove the key-value pair
                return True
        return False  # Key not found
# Example usage:
hash_table = HashTable()
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print(hash_table.search("name"))  # Output: Alice
print(hash_table.search("age"))   # Output: 30
hash_table.delete("name")
print(hash_table.search("name"))  # Output: None
```

Output

```
Alice
30
None
```

Explanation

We created a Hash Table data structure to store key value pairs using a hash function to determine the index.To handle collisions, we used chaining (storing multiple elements at the same index using a list).

We implemented insert, search, and delete operations to manage the data efficiently.